Problem Statement

In the fast-paced taxi booking sector, making the most of revenue is essential for long-term success and driver happiness.

Our goal is to use data-driven insights to **maximise revenue streams for taxi drivers** in order to meet this need. Our research aims to determine hether payment methods have an impact on fare pricing by focusing on the relationship between payment type and fare amount..

Objective

This project's main goal is to run an A/B test to examine the relationship between the total fare and the method of payment. We use python hypothesis testing and descritpive statistics to extract useful information that can help taxi drivers generate more cash. In particular we want to find out if there is a big difference in he fares for those who pay with credit cards verses those who pay with cash.

Research Question

Is there a relationship between total fare amount and payment type?

Can we nudge customers towards payment methods that generate higher revenue for drivers, without negatively impacting customer experience?

Importing libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
import scipy.stats as st

import warnings
warnings.filterwarnings('ignore')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
file_path = '/content/drive/MyDrive/Learning/DA-AB Testing/yellow_2024-04.csv'
```

```
df = pd.read_csv(file_path)
```

Data Description

In this dataset, we are considering only the Yellow Taxis Data, for the month of April 2024.

Key features included in the dataset are as follows:

- **VendorID** - A code indicating the TPEP provider that provided the record.

1. Creative Mobile Technologies
2. VeriFone Inc.

- **tpep_pickup_datetime** - The date and time when the meter was engaged.
- **tpep_dropoff_datetime** - The date and time when the meter was disengaged.
- **Passenger_count** - The number of passengers in the vehicle. This is a driver-entered value.
- **Trip_distance** - The elapsed trip distance in miles reported by the taximeter.
- **Pickup_longitude** - Longitude where the meter was engaged.
- **Pickup_latitude** - Latitude where the meter was engaged.
- **RateCodeID**- The final rate code in effect at the end of the trip.

1. Standard rate
2. JFK
3. Newark
4. Nassau or Westchester
5. Negotiated fare
6. Group ride

- **Store_and_fwd_flag** - This flag indicates whether the trip record was held in vehicle memory before sending to the vendor, aka "store and forward," because the vehicle did not have a connection to the server. Y= store and forward trip N= not a store and forward trip
- **Dropoff_longitude** - Longitude where the meter was disengaged.
- **Dropoff_ latitude** - Latitude where the meter was disengaged.
- **Payment_type** - A numeric code signifying how the passenger paid for the trip.

1. Credit card
2. Cash
3. No charge
4. Dispute
5. Unknown
6. Voided trip

- **Fare_amount** - The time-and-distance fare calculated by the meter.
- **Extra** - Miscellaneous extras and surcharges. Currently, this only includes. the 0.50 and 1 rush hour and overnight charges.
- **MTA_tax** - 0.50 MTA tax that is automatically triggered based on the metered rate in use.
- **Improvement_surcharge** - 0.30 improvement surcharge assessed trips at the flag drop. the improvement surcharge began being levied in 2015.
- **Tip_amount** - Tip amount – This field is automatically populated for credit card tips.Cash tips are not included.
- **Tolls_amount** - Total amount of all tolls paid in trip.
- **Total_amount** - The total amount charged to passengers. Does not include cash tips.
- **congestion_surcharge** - Total amount collected in trip for NYS congestion surcharge
- **Airport_fee** - 1.25 fee for airport pick-ups.

```
df.head()
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PUL
0	1	01-04-2024 00:02	01-04-2024 00:30	0	5.20	1	N	
1	2	01-04-2024 00:41	01-04-2024 00:55	1	5.60	1	N	
2	2	01-04-2024 00:48	01-04-2024 01:05	1	3.55	1	N	
3	2	01-04-2024 00:56	01-04-2024 01:05	1	1.06	1	N	
4	1	01-04-2024 00:08	01-04-2024 00:10	1	0.70	1	N	

```
df.tail()
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_fla
1048570	2	11-04-2024 09:17	11-04-2024 09:27	1	0.76	1	
1048571	2	11-04-2024 09:35	11-04-2024 09:52	1	2.05	1	
1048572	2	11-04-2024 09:53	11-04-2024 10:04	1	0.98	1	
1048573	2	11-04-2024 09:37	11-04-2024 09:43	1	0.60	1	
1048574	2	11-04-2024 09:07	11-04-2024 09:12	1	0.58	1	

Exploratory Data Analysis

```
df.shape
```

```
(1048575, 19)
```

```
df.columns
```

```
Index(['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime',
       'passenger_count', 'trip_distance', 'RatecodeID', 'store_and_fwd_flag',
       'PULocationID', 'DOLocationID', 'payment_type', 'fare_amount', 'extra',
       'mta_tax', 'tip_amount', 'tolls_amount', 'improvement_surcharge',
       'total_amount', 'congestion_surcharge', 'Airport_fee'],
      dtype='object')
```

```
df.describe()
```

	VendorID	passenger_count	trip_distance	RatecodeID	PULocationID	DOLocationID	payment_type	fare_amount	
count	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06	1.
mean	1.750500e+00	1.331478e+00	3.607193e+00	2.409688e+00	1.658168e+02	1.651806e+02	1.220818e+00	1.890238e+01	1.
std	4.327236e-01	8.431078e-01	1.458964e+02	1.133134e+01	6.308888e+01	6.920730e+01	5.424605e-01	1.910343e+01	1.
min	1.000000e+00	0.000000e+00	0.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	-7.184000e+02	-7.
25%	2.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.320000e+02	1.140000e+02	1.000000e+00	8.600000e+00	0.
50%	2.000000e+00	1.000000e+00	1.700000e+00	1.000000e+00	1.620000e+02	1.620000e+02	1.000000e+00	1.280000e+01	1.
75%	2.000000e+00	1.000000e+00	3.200000e+00	1.000000e+00	2.340000e+02	2.340000e+02	1.000000e+00	2.120000e+01	2.
max	2.000000e+00	9.000000e+00	1.342984e+05	9.900000e+01	2.650000e+02	2.650000e+02	5.000000e+00	8.731000e+02	1.

```
df.dtypes
```

	0
VendorID	int64
tpep_pickup_datetime	object
tpep_dropoff_datetime	object
passenger_count	int64
trip_distance	float64
RatecodeID	int64
store_and_fwd_flag	object
PULocationID	int64
DOLocationID	int64
payment_type	int64
fare_amount	float64
extra	float64
mta_tax	float64
tip_amount	float64
tolls_amount	float64
improvement_surcharge	float64
total_amount	float64
congestion_surcharge	float64
Airport_fee	float64

dtype: object

Datetime Conversion

```
df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'], dayfirst=True)
df['tpep_dropoff_datetime'] = pd.to_datetime(df['tpep_dropoff_datetime'],dayfirst=True)
```

```
df.dtypes
```

	0
VendorID	int64
tpep_pickup_datetime	datetime64[ns]
tpep_dropoff_datetime	datetime64[ns]
passenger_count	int64
trip_distance	float64
RatecodeID	int64
store_and_fwd_flag	object
PULocationID	int64
DOLocationID	int64
payment_type	int64
fare_amount	float64
extra	float64
mta_tax	float64
tip_amount	float64
tolls_amount	float64
improvement_surcharge	float64
total_amount	float64
congestion_surcharge	float64
Airport_fee	float64

dtype: object

Duration Feature Engineering

```
df['duration'] = df['tpep_dropoff_datetime']-df['tpep_pickup_datetime']
df['duration'] = df['duration'].dt.total_seconds()/60
```

```
df
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_fla
0	1	2024-04-01 00:02:00	2024-04-01 00:30:00	0	5.20	1	
1	2	2024-04-01 00:41:00	2024-04-01 00:55:00	1	5.60	1	
2	2	2024-04-01 00:48:00	2024-04-01 01:05:00	1	3.55	1	
3	2	2024-04-01 00:56:00	2024-04-01 01:05:00	1	1.06	1	
4	1	2024-04-01 00:08:00	2024-04-01 00:10:00	1	0.70	1	
...	...	...	...	...	...	...	
1048570	2	2024-04-11 09:17:00	2024-04-11 09:27:00	1	0.76	1	
1048571	2	2024-04-11 09:35:00	2024-04-11 09:52:00	1	2.05	1	
1048572	2	2024-04-11 09:53:00	2024-04-11 10:04:00	1	0.98	1	
1048573	2	2024-04-11 09:37:00	2024-04-11 09:43:00	1	0.60	1	
1048574	2	2024-04-11 09:07:00	2024-04-11 09:12:00	1	0.58	1	

1048575 rows × 20 columns

```
data = df[['passenger_count','payment_type','fare_amount','trip_distance','duration']]
```

```
data
```

	passenger_count	payment_type	fare_amount	trip_distance	duration
0	0	1	29.6	5.20	28.0
1	1	1	25.4	5.60	14.0
2	1	1	20.5	3.55	17.0
3	1	2	10.0	1.06	9.0
4	1	1	5.1	0.70	2.0
...	...	...	...	...	...
1048570	1	1	9.3	0.76	10.0
1048571	1	2	16.3	2.05	17.0
1048572	1	1	11.4	0.98	11.0
1048573	1	1	7.2	0.60	6.0
1048574	1	2	6.5	0.58	5.0

1048575 rows × 5 columns

Missing Values Check

```
data.isnull().sum()
```

	0
passenger_count	0
payment_type	0
fare_amount	0
trip_distance	0
duration	0

dtype: int64

```
data[data.duplicated()]
```

	passenger_count	payment_type	fare_amount	trip_distance	duration
270	1	1	11.4	1.72	10.0
352	1	1	6.5	0.80	5.0
363	1	1	8.6	1.34	6.0
429	1	1	5.8	0.63	3.0
442	1	2	6.5	0.90	5.0
...	...	...	...	...	...
1048570	1	1	9.3	0.76	10.0
1048571	1	2	16.3	2.05	17.0
1048572	1	1	11.4	0.98	11.0
1048573	1	1	7.2	0.60	6.0
1048574	1	2	6.5	0.58	5.0

730064 rows × 5 columns

```
data.drop_duplicates(inplace=True)
```

```
data.shape
```

```
(318511, 5)
```

Data.describe() (After Cleaning)

```
data.describe()
```

	passenger_count	payment_type	fare_amount	trip_distance	duration
count	318511.000000	318511.000000	318511.000000	318511.000000	318511.000000
mean	1.679823	1.397176	32.210529	7.613630	29.964918
std	1.182747	0.733103	27.209198	264.654514	68.236457
min	0.000000	1.000000	-718.400000	0.000000	-39.000000
25%	1.000000	1.000000	15.600000	2.000000	15.000000
50%	1.000000	1.000000	26.800000	4.390000	23.000000
75%	2.000000	2.000000	44.300000	9.930000	35.000000
max	9.000000	5.000000	873.100000	134298.400000	9767.000000

```
data['passenger_count'].value_counts(normalize=True)
```

passenger_count	proportion
1	0.613514
2	0.199120
3	0.071458
4	0.053659
5	0.025161
6	0.018831
0	0.018222
7	0.000016
8	0.000013
9	0.000006

dtype: float64

```
data['payment_type'].value_counts(normalize=True)
```

payment_type	proportion
1	0.707323
2	0.232683
4	0.044498
3	0.015494
5	0.000003

dtype: float64

```
data = data[data['payment_type'] < 3]
data = data[(data['passenger_count'] > 0)&(data['passenger_count'] < 6)]
```

```
data.shape
```

```
(287871, 5)
```

```
data['payment_type'].replace([1,2],['Card','Cash'], inplace=True)
```

```
data.describe()
```

	passenger_count	fare_amount	trip_distance	duration
count	287871.000000	287871.000000	287871.000000	287871.000000
mean	1.640044	34.537961	7.978845	31.127224
std	1.025272	25.510115	278.374792	70.322032
min	1.000000	-718.400000	0.000000	-39.000000
25%	1.000000	17.700000	2.200000	16.000000
50%	1.000000	28.200000	4.740000	24.000000
75%	2.000000	46.400000	10.250000	36.000000
max	5.000000	873.100000	134298.400000	9767.000000

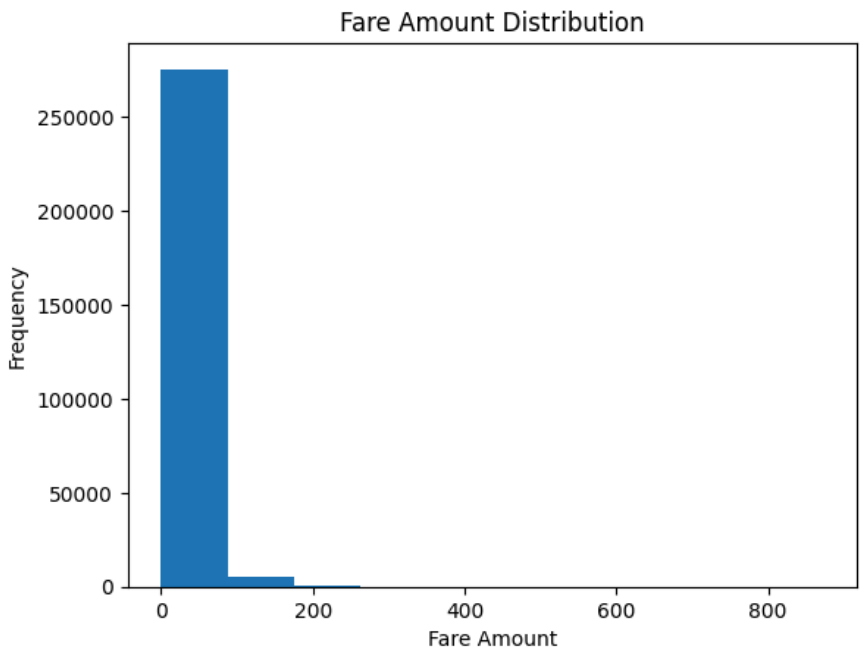
Remove Negative Values

```
# remove negative values in fare amount, trip distance and duration column
data = data[data['fare_amount']>0]
data = data[data['trip_distance']>0]
data = data[data['duration']>0]
```

```
data.describe()
```

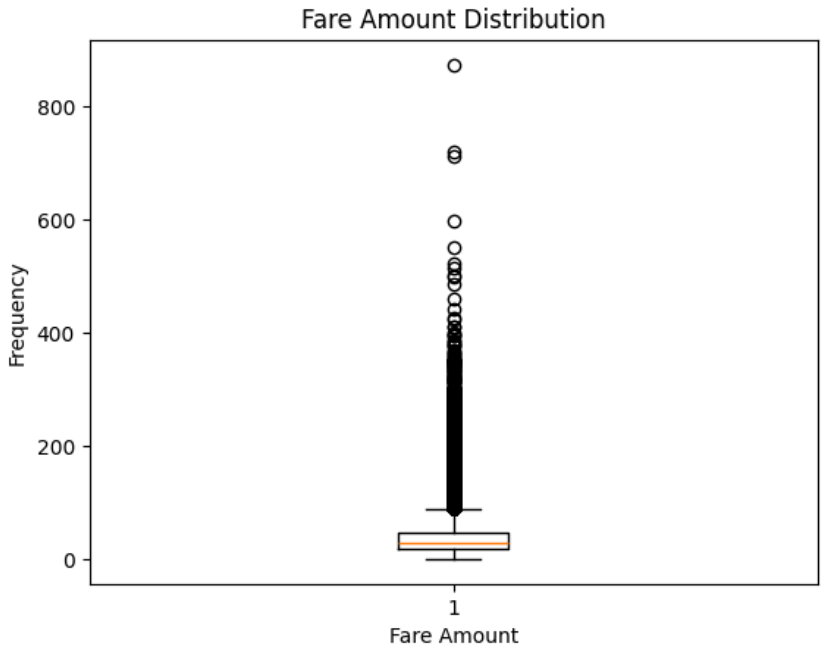
	passenger_count	fare_amount	trip_distance	duration
count	281357.000000	281357.000000	281357.000000	281357.000000
mean	1.642397	34.956211	8.118124	31.406100
std	1.027328	24.343542	281.576410	68.836393
min	1.000000	0.010000	0.010000	1.000000
25%	1.000000	17.700000	2.300000	16.000000
50%	1.000000	28.200000	4.880000	25.000000
75%	2.000000	46.400000	10.360000	37.000000
max	5.000000	873.100000	134298.400000	9767.000000

```
plt.hist(data['fare_amount'])
plt.xlabel('Fare Amount')
plt.ylabel('Frequency')
plt.title('Fare Amount Distribution')
plt.show()
```




```
plt.boxplot(data['fare_amount'])
plt.xlabel('Fare Amount')
plt.ylabel('Frequency')
plt.title('Fare Amount Distribution')
```

```
plt.show()
```



Outlier Removal (IQR Method)

```
for col in ['fare_amount','trip_distance','duration']:
  q1 = data[col].quantile(0.25)
  q3 = data[col].quantile(0.75)
  IQR = q3 - q1

  lower_bound = q1 - 1.5*IQR
  upper_bound = q3 + 1.5*IQR
  data = data[(data[col] >= lower_bound) & (data[col] <= upper_bound)]
```

```
data
```

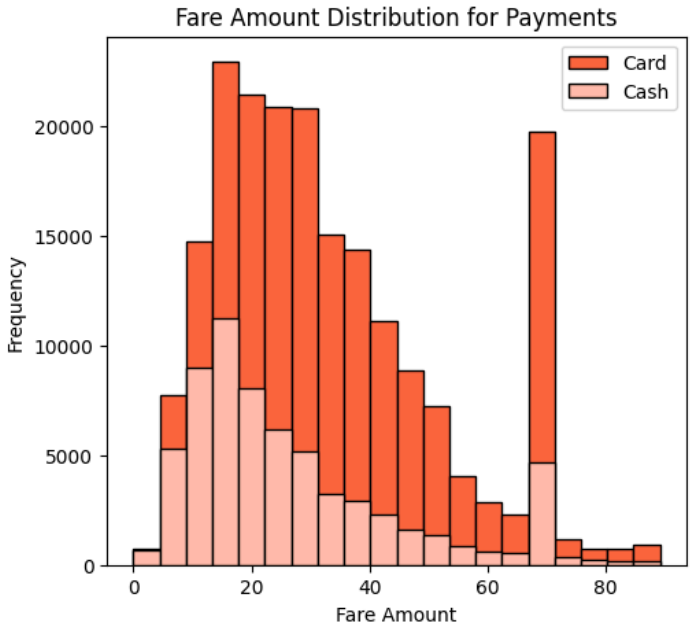
	passenger_count	payment_type	fare_amount	trip_distance	duration
1	1	Card	25.4	5.60	14.0
2	1	Card	20.5	3.55	17.0
3	1	Cash	10.0	1.06	9.0
4	1	Card	5.1	0.70	2.0
5	1	Cash	8.6	1.63	4.0
...	...	...	...	...	...
1048523	1	Card	31.7	4.10	40.0
1048543	1	Card	59.0	11.60	54.0
1048544	1	Card	14.2	1.10	14.0
1048550	1	Card	46.4	10.59	38.0
1048560	4	Card	26.8	4.18	25.0

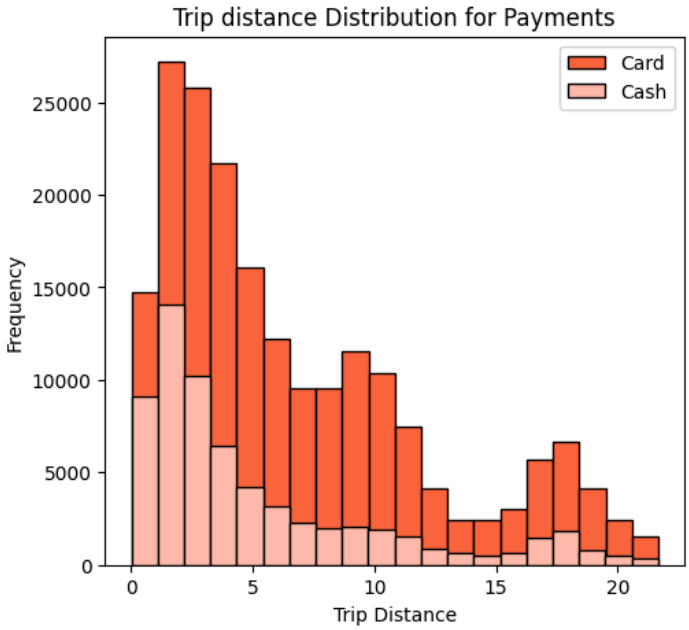
263242 rows × 5 columns

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)

plt.hist(data[data['payment_type'] == 'Card']['fare_amount'],  histtype='barstacked', bins=20, edgecolor = 'k', color = '#FA6
plt.hist(data[data['payment_type'] == 'Cash']['fare_amount'],  histtype='barstacked', bins=20, edgecolor = 'k', color = '#FFE
plt.xlabel('Fare Amount')
plt.ylabel('Frequency')
plt.legend(['Card','Cash'])
plt.title('Fare Amount Distribution for Payments')

plt.subplot(1,2,2)

plt.hist(data[data['payment_type'] == 'Card']['trip_distance'],  histtype='barstacked', bins=20, edgecolor = 'k', color = '#F
plt.hist(data[data['payment_type'] == 'Cash']['trip_distance'],  histtype='barstacked', bins=20, edgecolor = 'k', color = '#F
plt.xlabel('Trip Distance')
```

```
plt.ylabel('Frequency')
plt.legend(['Card','Cash'])
plt.title('Trip distance Distribution for Payments')
```

```
Text(0.5, 1.0, 'Trip distance Distribution for Payments')
```

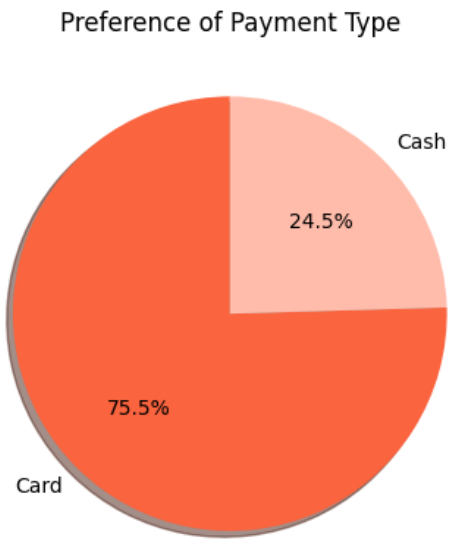





```
data.groupby('payment_type').agg({'fare_amount':['mean','std'],'trip_distance':['mean','std']})
```

	fare_amount		trip_distance	
	mean	std	mean	std
payment_type				
Card	33.445392	19.060525	6.754946	5.368844
Cash	27.165547	18.661152	5.185163	5.073176

```
plt.title('Preference of Payment Type')
plt.pie(data['payment_type'].value_counts(normalize=True),
        labels=data['payment_type'].value_counts().index,
        startangle=90,
        shadow=True,
        colors=['#FA643F','#FFBCAB'],
        autopct='%1.1f%%')
plt.show()
```




```
passenger_count = data.groupby(['payment_type','passenger_count'])[['passenger_count']].count()
passenger_count.rename(columns = {'passenger_count':'count'}, inplace=True)
passenger_count.reset_index(inplace=True)
```

```
passenger_count
```

	payment_type	passenger_count	count
0	Card	1	125037
1	Card	2	41880
2	Card	3	15154
3	Card	4	10380
4	Card	5	6187
5	Cash	1	39580
6	Cash	2	13953
7	Cash	3	5120
8	Cash	4	4420
9	Cash	5	1531

Next steps: Generate code with passenger_count New interactive sheet

```
passenger_count['perc'] = passenger_count['count']/passenger_count['count'].sum()*100
```

```
passenger_count
```

	payment_type	passenger_count	count	perc
0	Card	1	125037	47.498879
1	Card	2	41880	15.909315
2	Card	3	15154	5.756680
3	Card	4	10380	3.943140
4	Card	5	6187	2.350309
5	Cash	1	39580	15.035595
6	Cash	2	13953	5.300446
7	Cash	3	5120	1.944978
8	Cash	4	4420	1.679063
9	Cash	5	1531	0.581594

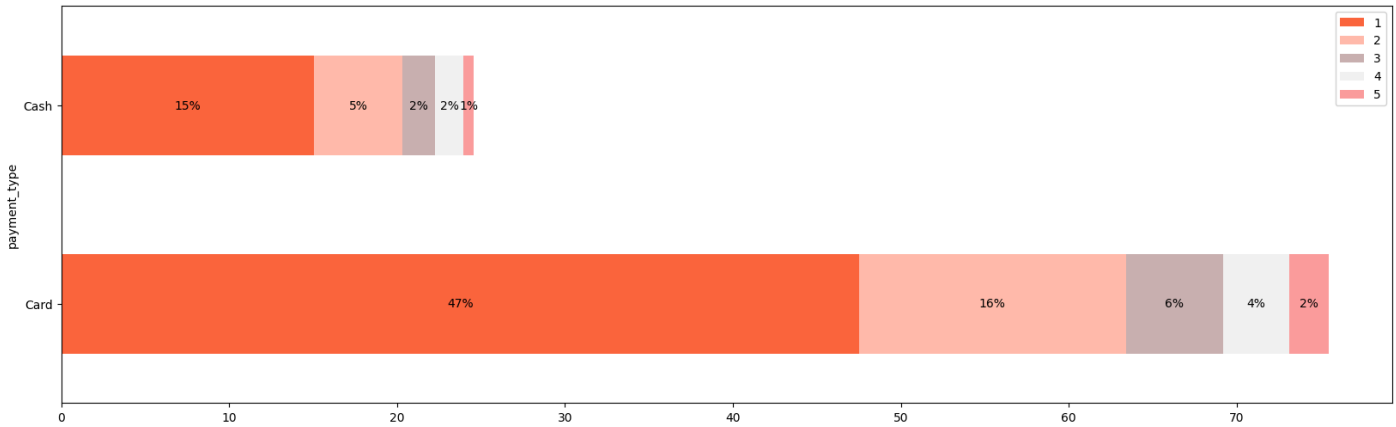
Next steps: Generate code with passenger_count New interactive sheet

```
cust_df = pd.DataFrame(columns = ['payment_type',1,2,3,4,5])
cust_df['payment_type'] = ['Card','Cash']
cust_df.iloc[0, 1:] = passenger_count.iloc[0:5,-1]
cust_df.iloc[1, 1:] = passenger_count.iloc[5:, -1]
cust_df
```

	payment_type	1	2	3	4	5
0	Card	47.498879	15.909315	5.75668	3.94314	2.350309
1	Cash	15.035595	5.300446	1.944978	1.679063	0.581594

Next steps: Generate code with cust_df New interactive sheet

```
fig, ax = plt.subplots(figsize = (20,6))
cust_df.plot(x='payment_type', kind = 'barh', stacked = True, ax = ax, color = ['#FA643F','#FFBCAB','#CBB2B2','#F1F1F1','#F

# add percentage text
for p in ax.patches:
    width = p.get_width()
    height = p.get_height()
    x, y = p.get_xy()
    ax.text(x + width /2,
            y + height /2,
            '{:.0f}%'.format(width),
            horizontalalignment = 'center',
            verticalalignment = 'center')
```



Passenger Count Analysis

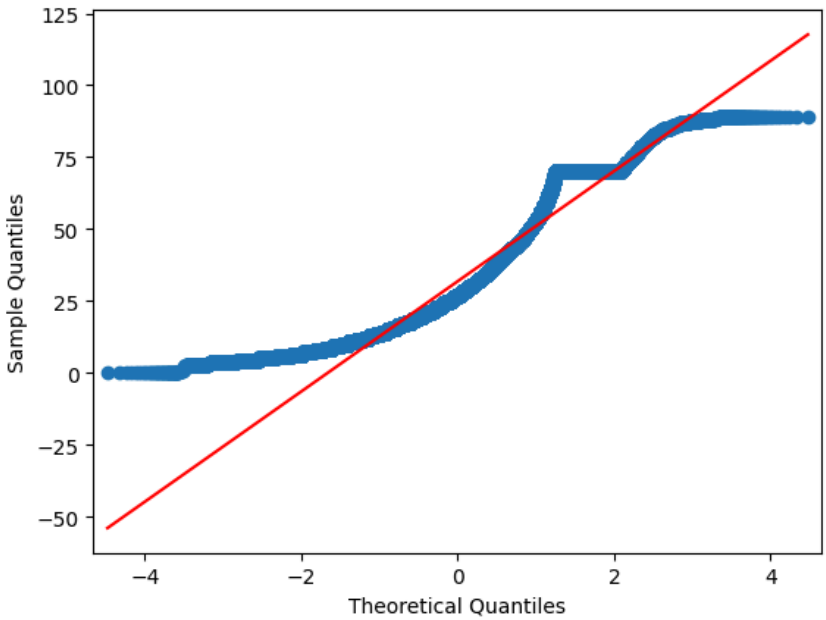
Among card payments, rides with a_single passenger (passenger_count== 1) comprise-the largest proportion; constituting 47% of all card transactions. . Similarly, cash payments are predominantly associated with single-passenger rides, making up 15% of all cash transactions. There is a noticeable decrease in the percentage of transactions as the passenger count increases, suggesting that larger groups are less likely to use taxis or may opt for alternative payment methods. • These insights emphasize the importance of considering both payment method and passenger count when analyzing transaction data, as they provide valuable insights into cus tomer behavior and preferences.

Hypothesis Testing

Null Hypothesis : There is no difference in average fare between customers who use credit cards and customers who use cash.

Alternative Hypothesis : There is a difference in average fare between customers who use credit cards and costomers who use cash.

```
sm.qqplot(data['fare_amount'], line = 's')
plt.show()
```




```
card_sample = data[data['payment_type'] == 'Card']['fare_amount']
cash_sample = data[data['payment_type'] == 'Cash']['fare_amount']
```

```
t_stats, p_value = st.ttest_ind(a = card_sample, b = cash_sample, equal_var = False)
print('t-statistic: ', t_stats)
print('p-value: ', p_value)
```

```
t-statistic:  73.90953600403142
p-value:  0.0
```

The independent two-sample t-test produced a very high t-statistic (73.91) and a p-value effectively equal to zero, which is far below the significance level of 0.05. Therefore, we reject the null hypothesis and conclude that there is a statistically significant difference in average fare amounts between customers who pay by card and those who pay by cash.

Further analysis shows that card payments are associated with higher average fares compared to cash payments. This indicates that payment method has a meaningful impact on revenue, and encouraging customers to use card payments could potentially increase overall earnings for taxi drivers.

```
Start coding or generate with AI.
```